



FlyBlox MarketPlace Audit Report

Version 1.0

Published by Dionis Xhaferi

5/22/25

Findings Summary

ID	Title	Severity
1	Reentrancy Vulnerability in Transfers	High
2	Missing nonReentrant Guards	Medium
3	No Pause Mechanism for Emergency	Low
3	ERC-20 Transfer Return Value Not Checked	Low
3	Fee Calculation May Truncate to Zero	Low
4	Lack of Event for Token List Updates	Informational
5	Fee Misalignment Between Order & System	Informational
6	Unbounded Gas Consumption in Loop	Informational

[H-1] Reentrancy Vulnerability in Transfers

Description:

Several functions in the MarketplacePayments contract perform external token or ETH transfers before updating the internal state. This introduces a classic reentrancy vulnerability where a malicious contract can re-enter the function (or another vulnerable function) and exploit inconsistent or unprotected contract state.

Impact:

High. A malicious contract acting as a seller (or buyer in some flows) could re-enter the contract during the ETH/token transfer and:

- Call another function like `returnFundsToBuyer()` before the state is changed to Completed.
- Drain funds or alter order state multiple times.
- Cause financial loss and critical inconsistencies.

Proof of Code:

```
// Line in markCompleteAndReleaseFundsToSeller
payable(order.seller).transfer(order.amount - feeValue); // vulnerable
position
order.state = State.Completed; // should be before transfer
```

Recommended Mitigation: Move state updates before external calls and use a reentrancy guard.

```
order.state = State.Completed;
payable(order.seller).transfer(order.amount - feeValue);
```

[M-1] Missing nonReentrant Guards

Description:

Public functions handling ETH and token transfers are not protected with reentrancy guards.

Impact:

Medium. Increases risk of reentrancy exploit.

Proof of Code: All functions like:

```
function markCompleteAndReleaseFundsToSeller(...) public payable
```

Recommended Mitigation: Inherit and apply OpenZeppelin's ReentrancyGuard:

```
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
contract MarketplacePayments is Ownable, ReentrancyGuard {
    ...
    function markCompleteAndReleaseFundsToSeller(...) public payable nonReentrant
    {
```

4. [L-1] No Pause Mechanism for Emergency

Description:

No way to halt the contract in case of an emergency.

Impact:

Low. Can't freeze operations if critical bug/exploit found.

Proof of Code:

```
// No use of Pausable or emergency halt mechanism
```

Recommended Mitigation: Use OpenZeppelin's Pausable contract.

```
import "@openzeppelin/contracts/security/Pausable.sol";
contract MarketplacePayments is Ownable, Pausable {
    function createAndDeposit(...) public whenNotPaused { ... }
}
```

[L-2] ERC-20 Transfer Return Value Not Checked

Description:

The contract assumes all ERC-20 transfer and transferFrom calls return true, but some tokens (e.g., USDT) do not comply strictly and may revert or return nothing.

Impact:

Medium. Transfers may silently fail, causing deposits or payouts to fail unexpectedly.

Proof of Code:

```
IERC20(order.paymentToken).transfer(order.seller, order.amount - feeValue);
```

Recommended Mitigation: Use OpenZeppelin's SafeERC20 library:

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
using SafeERC20 for IERC20;
// Then use:
IERC20(order.paymentToken).safeTransfer(order.seller, order.amount -
feeValue);
```

[L-3] Fee Calculation May Truncate to Zero

Description:

For small order values or fees, the calculated fee may round to 0 due to integer division.

Impact:

Low. Could result in no fees being collected on small orders.

Proof of Code:

```
uint256 feeValue = (order.amount * fee) / 10_000;
```

Recommended Mitigation: Ensure a minimum fee or warn:

```
require(feeValue > 0, "Fee too small");
```

[I-1] Lack of Event for Token List Updates

Description:

updateTokensList() does not emit any event.

Impact:

Informational. Harder for off-chain indexers to track changes.

Proof of Code:

```
function updateTokensList(...) public onlyOwner returns (bool)
```

Recommended Mitigation:

 Emit event:

```
event TokenListUpdated(address token, bool allowed);  
emit TokenListUpdated(_tokenAddress, allowed);
```

[I-2] Fee Misalignment Between Order & System

Description:

Order stores fee at creation, but it's never used in later logic – contract uses the global fee.

Impact:

Informational. Potential inconsistency if fee is changed post-order.

Proof of Code:

```
order.fee = fee; // stored but unused
```

Recommended Mitigation:

 Either remove from order or use it in fee calculations.

[I-3] Unbounded Gas Consumption in Loops

Description:

While current implementation doesn't explicitly loop through mappings, future extensions (e.g., refunds to multiple addresses) might hit gas limits.

Impact:

Low. Latent issue; not currently exploitable.

Proof of Code:

 N/A

Recommended Mitigation:

 Avoid unbounded loops. Use pagination or off-chain tracking.

End of Report